

# Understanding Language Models

by

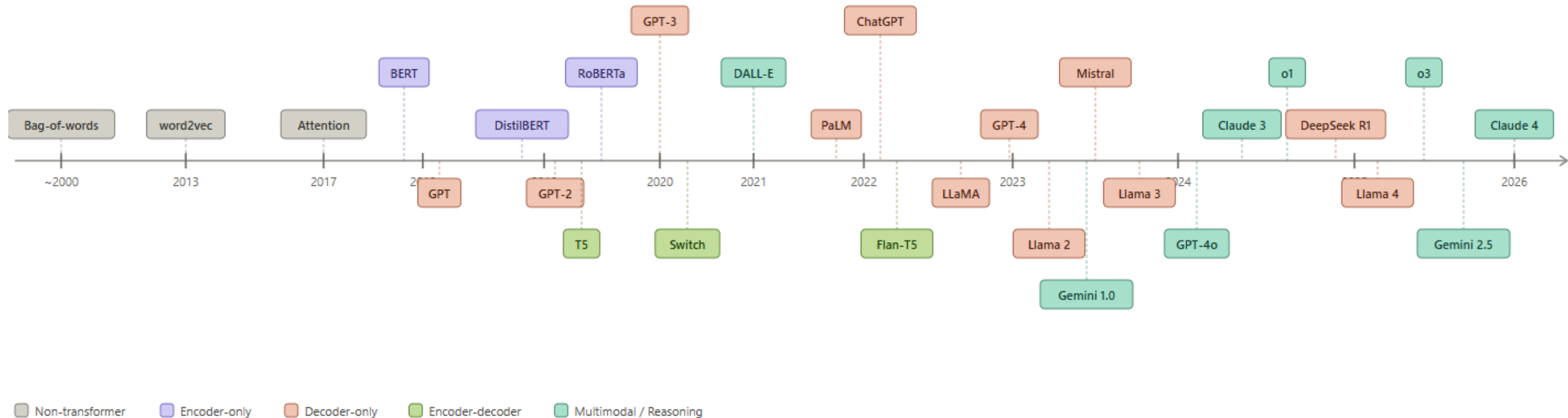
Rajdip Nayek

Assistant Professor,  
Applied Mechanics Department,  
IIT Delhi

Instructor email: [rajdipn@am.iitd.ac.in](mailto:rajdipn@am.iitd.ac.in)

# From Words to GPT: How Large Language Models Work

- Every day we interact with systems that understand language—Google Search, ChatGPT, Copilot, Claude.

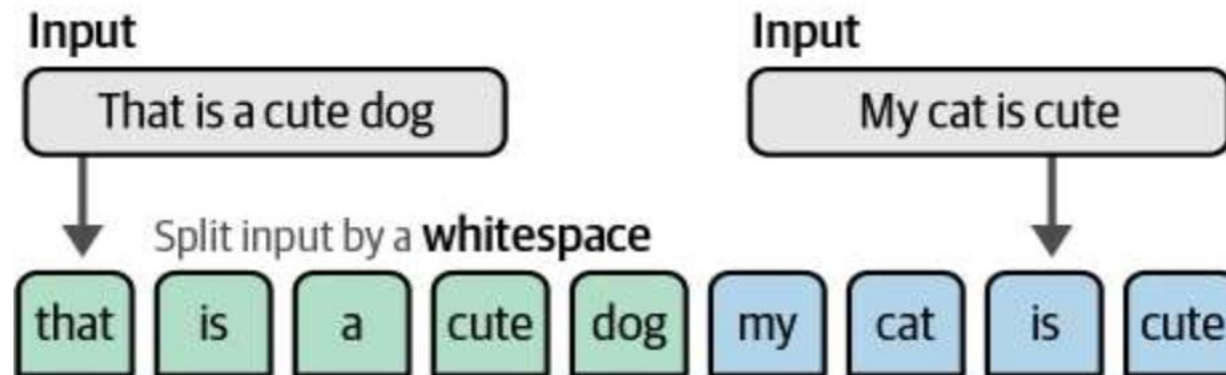


# What is Natural Language Processing (NLP)?

- How did computers go from storing text as characters to generating human-like responses?
- NLP (or **Language AI**) refers to a subfield of AI that focuses on developing technologies capable of understanding, processing, and generating human language
- The fundamental challenge:
  - Humans understand meaning.
  - Computers only understand numbers.
- The central challenge of NLP is converting language into numbers without losing meaning

# Converting Text into Numbers

- Bag-of-words was first mentioned in around 1950s but became popular in 2000s.
- Computers cannot directly process text
- We first split a sentence into smaller pieces individual words or sub-words called **tokens**
- This process is called *tokenization*
- The most common method for tokenization is by splitting on a whitespace to create individual words.

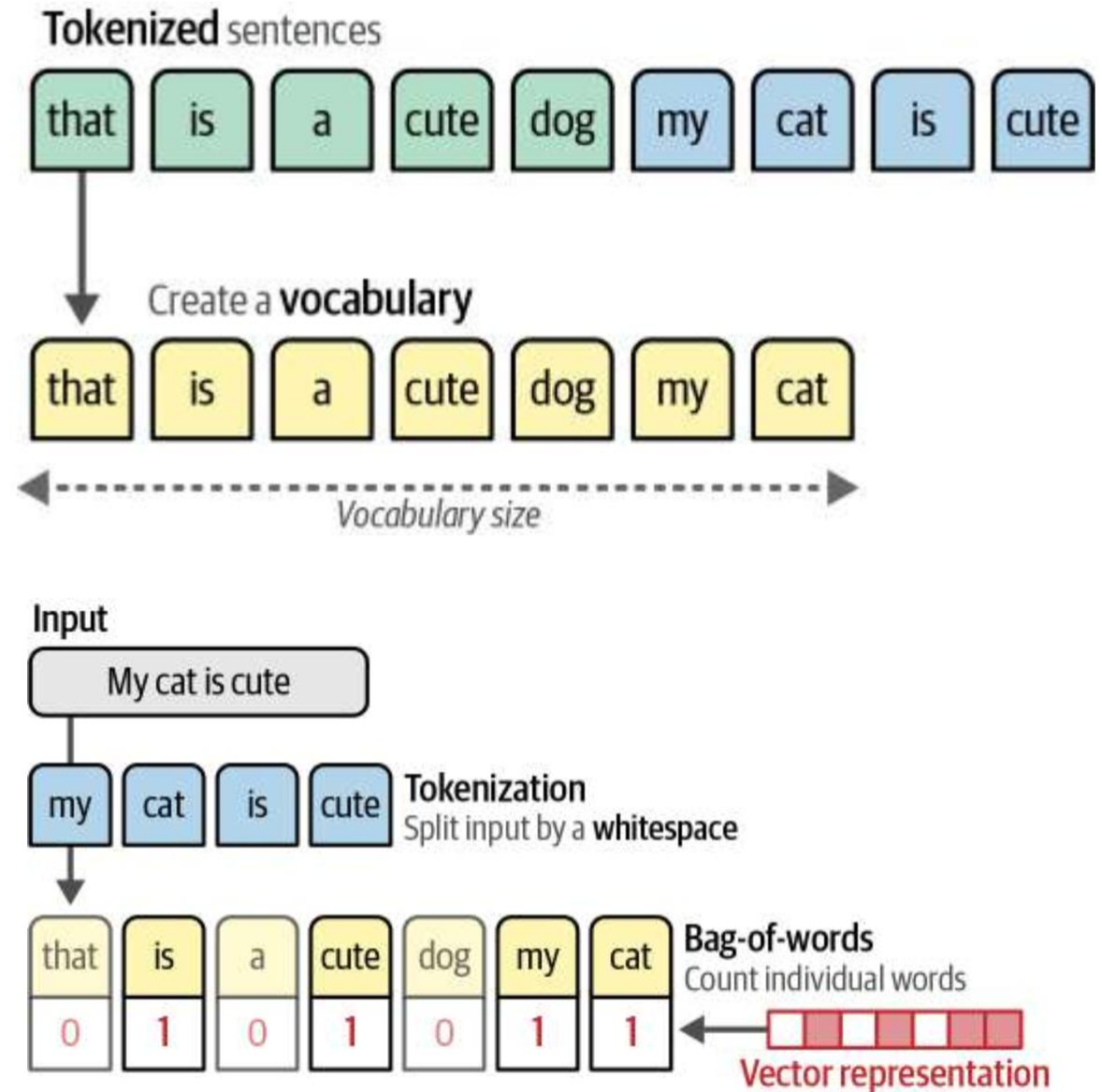


**Token** = the basic unit processed by an NLP model

# Bag-of-Words Representation

- Create a **vocabulary** of all unique words.
- Using our **vocabulary**, count how many times each word appears
- Represent the sentence as a vector of counts, also called **vector representations**.

✗ Word order is lost.



# Word2vec (2013): Learning meaning from context

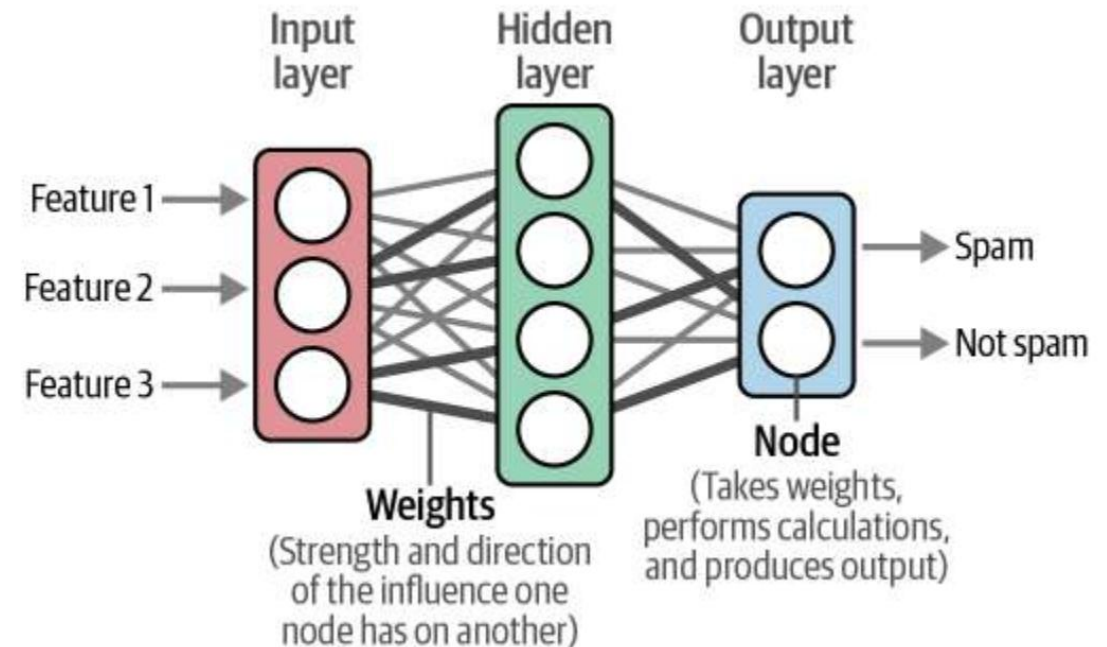
- **Difficulty:** Bag-of-Words captures words but it completely ignores the meaning
  - Examples: “car” and “automobile” are treated as unrelated as “bank”
- **Goal:** Convert words into numbers while preserving meaning.
- **Word2Vec** (2013) introduced the idea of **embeddings**.
  - Words that appear in similar contexts tend to have similar meanings

dog → bark, pet, puppy  
cat → meow, pet, kitten

$\text{embedding}(\text{dog}) \approx \text{embedding}(\text{cat})$

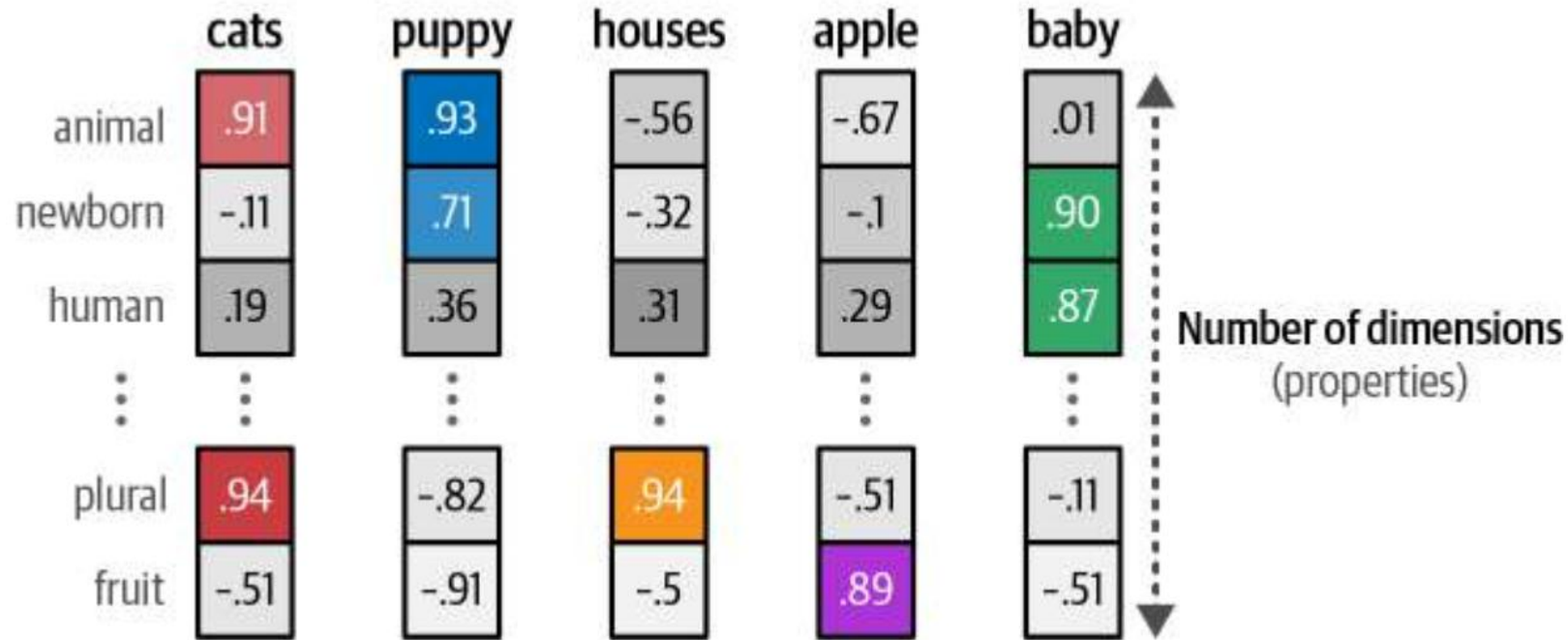
## How does word2vec learn embeddings?

- Train a neural network on a very large text corpus (Wikipedia, books, web pages)
- The model learns which words frequently appear near one another
- During training, each word acquires a *dense* vector representation called an **embedding**



# Word2vec (2013) embeddings of words

- Example:



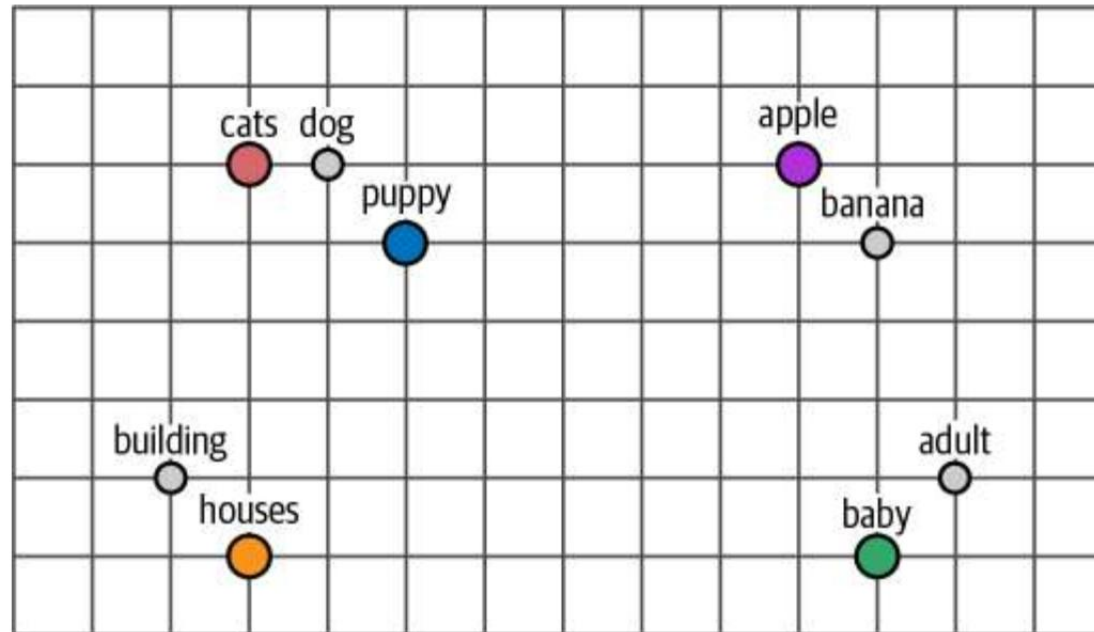
*The values of embeddings represent properties that are used to represent words. We may oversimplify by imagining that dimensions represent concepts (which they don't), but it helps express the idea*

- In practice, these properties are often quite obscure and seldom relate to a single entity or humanly identifiable concept
- However, together, these properties make sense to a computer and serve as a good way to translate human language into computer language.

# Why do Word **Embeddings** matter?

- An **embedding** is a dense vector representation of a word
- Each word is represented by dozens or hundreds of numbers (large vectors)
- Similar meanings → nearby vectors      Different meanings → distant vectors
- This allows computers to perform **semantic search** and **similarity matching**

*Embeddings of words that are similar will be close to each other in dimensional space*





# Words change meaning with Context

- Example:

I deposited money in the bank.

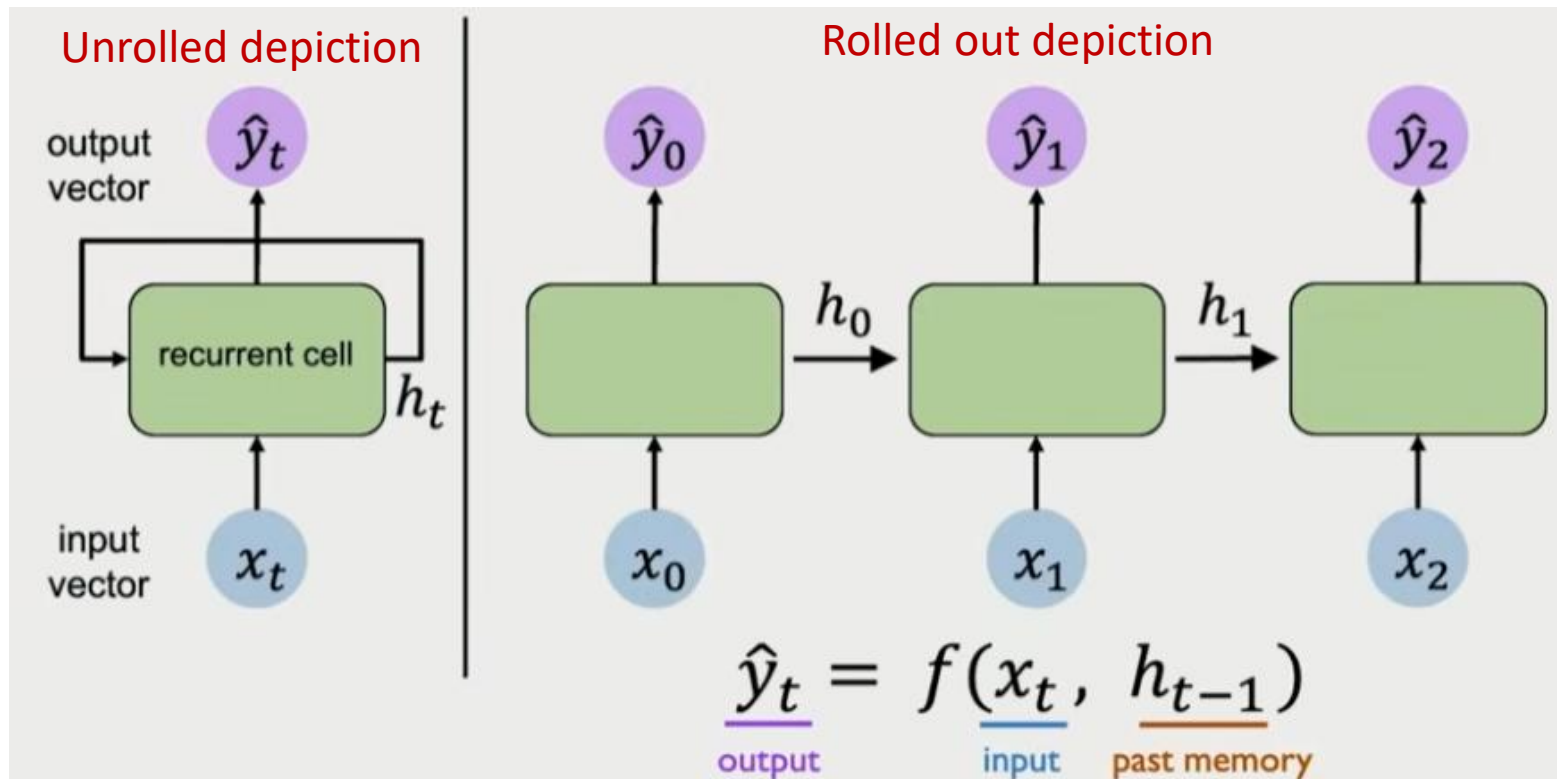
VS

I sat on the river bank.

- Word2Vec gives “bank” the same embedding in both sentences
- Humans understand the difference because of context
- Models must represent words differently depending on surrounding words
- A step in encoding the context was achieved through **Recurrent Neural Networks** (RNNs)
- To do so, RNNs are used for two tasks, *encoding* or representing an input sentence and *decoding* or generating an output sentence.

# Recurrent Neural Networks (RNN) recap

- Model structure of RNN



$h_t$ : Also called hidden cell states

RNNs use the concept of **hidden states** to store information from past

# Recurrent Neural Networks (RNN) recap

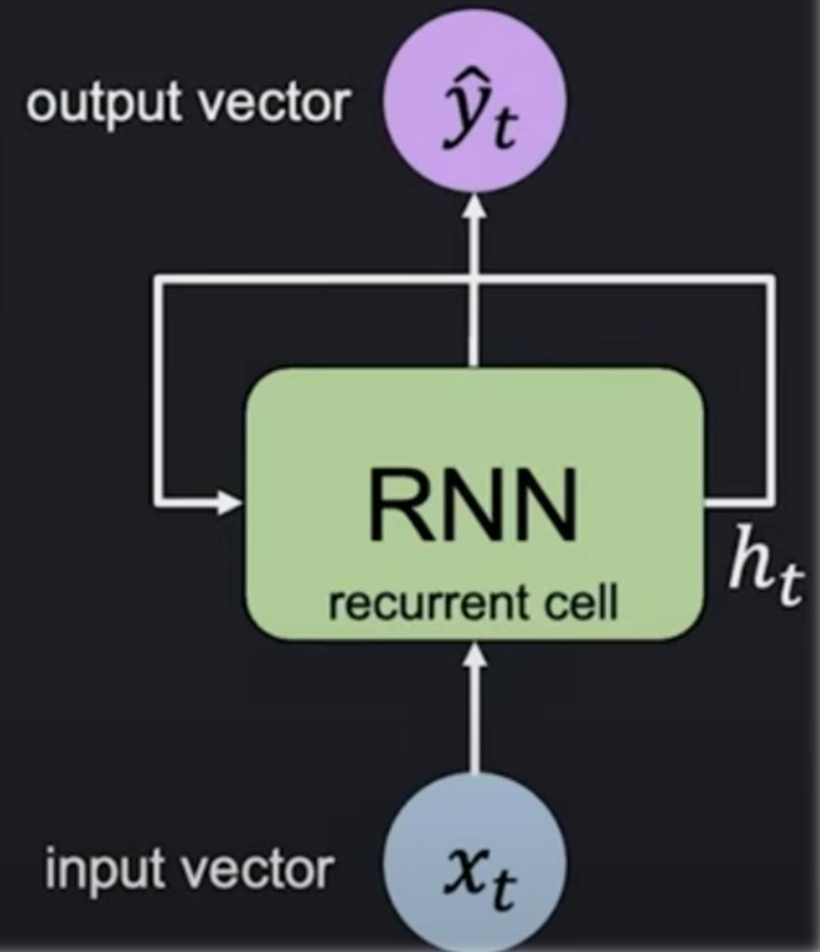
## Pseudo code

```
my_rnn = RNN()
hidden_state = [0, 0, 0, 0]

sentence = ["I", "love", "recurrent", "neural"]

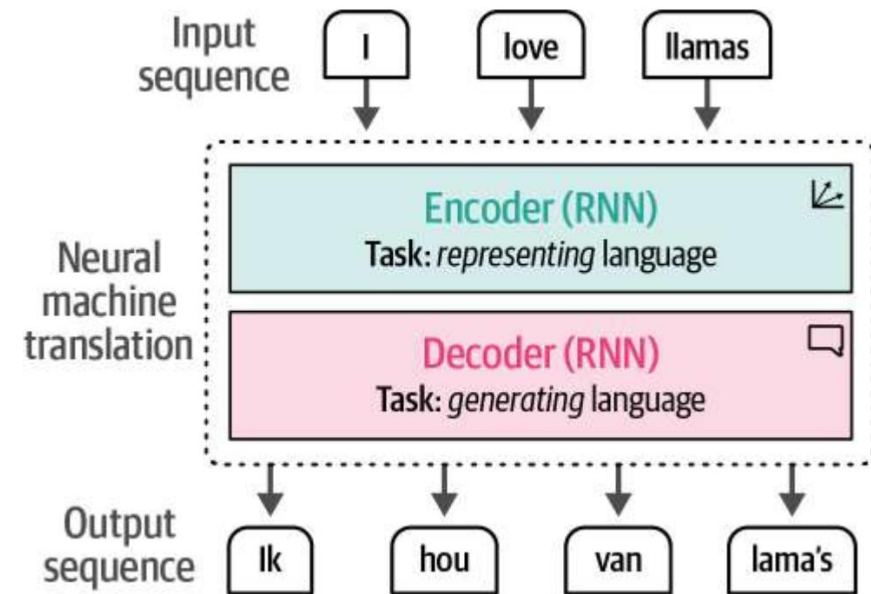
for word in sentence:
    prediction, hidden_state = my_rnn(word, hidden_state)

next_word_prediction = prediction
# >>> "networks!"
```



# Encoder-Decoder RNNs

- RNNs process **one word at a time**, *sequentially*.
- Earlier words influence later words (**have memory**).
- **Encoder**: Understands the input sentence.
- **Decoder**: Generates the output sentence.



*Two recurrent neural networks (decoder and encoder) translating an input sequence from English to Dutch*

RNNs introduced context awareness

# Where RNNs struggled

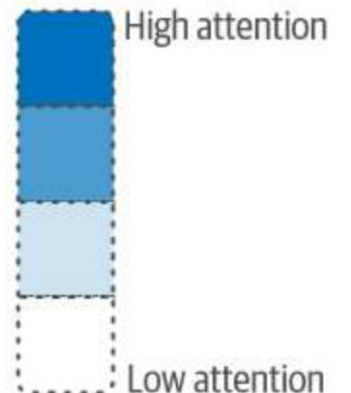
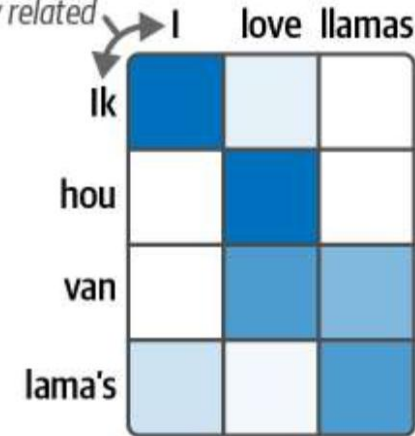
- The encoder compresses the entire sentence into a **single context vector**.
- Important information (at the beginning of a long sentence) may be lost
- We need a mechanism that can directly focus on important words.

## Attention: Learn What Matters

- Instead of using only one context vector:
  - ✓ The decoder can directly look at all input words
- When generating a new word, the model learns:

Which input words are most relevant right now?

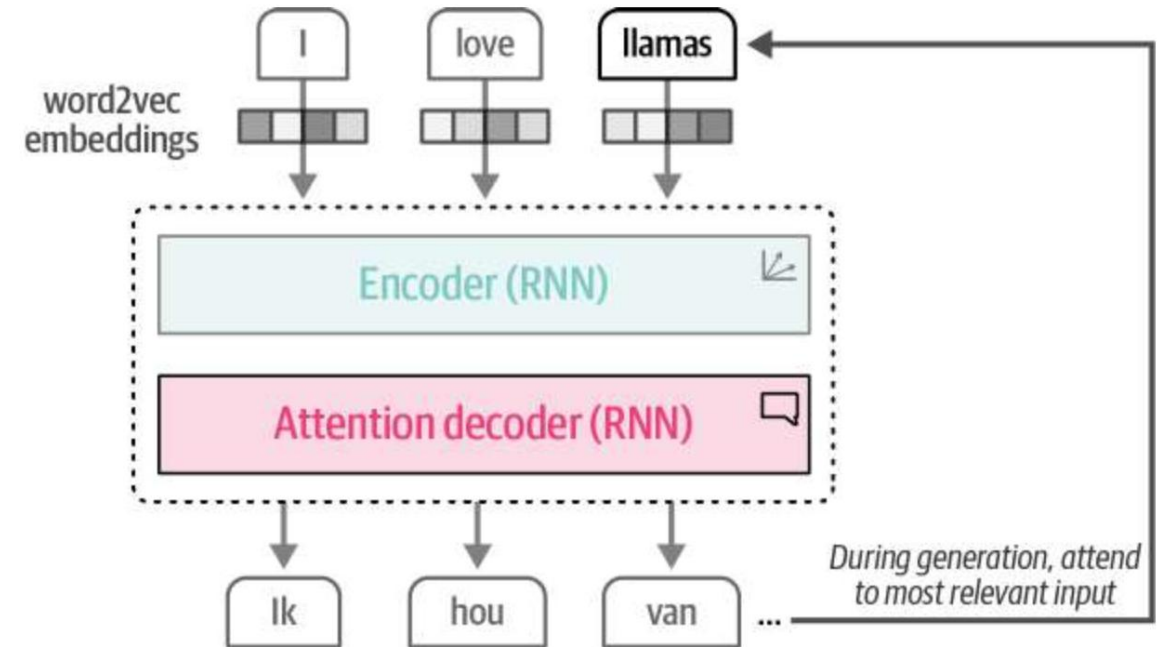
Words with similar meaning  
have higher attention weights  
since they are highly related



Attention allowed the decoder to dynamically focus on relevant parts of the input sequence

# Encoding and Decoding Context with Attention

- Attention allows the decoder to **focus on the most relevant input words**.
- Different output words can attend to different parts of the input sentence.
- This improves translation quality and contextual understanding.
- Training remains sequential and difficult to parallelize.

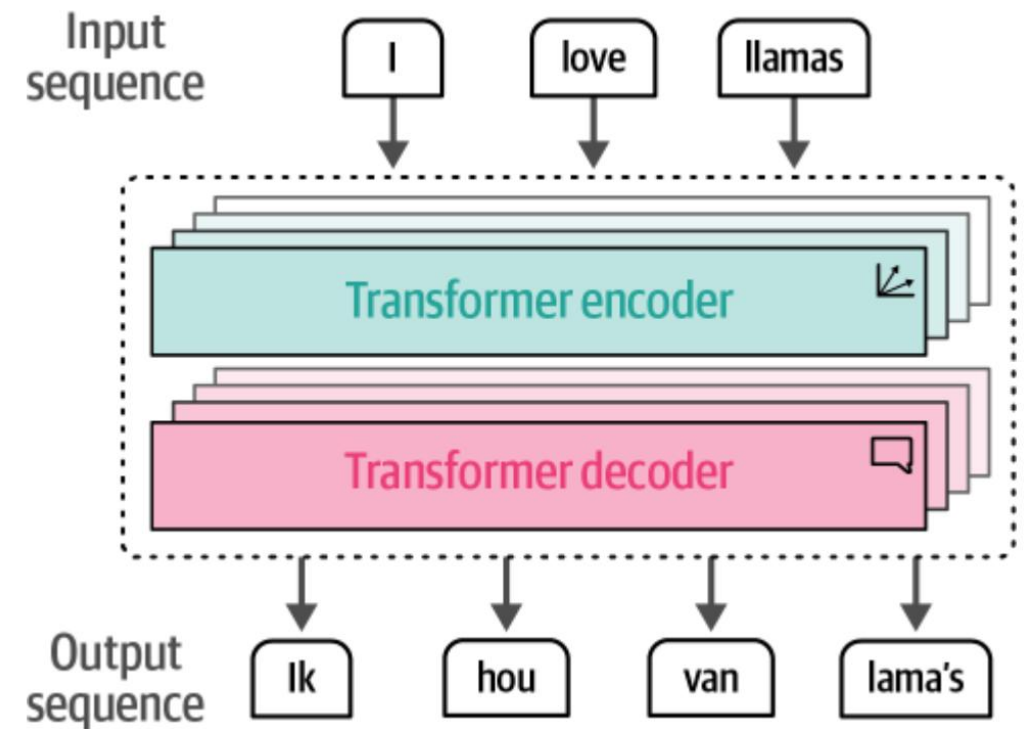


*After generating the words "Ik," "hou," and "van," the attention mechanism of the decoder enables it to focus on the word "llamas" before it generates the Dutch translation ("lama's").*

**Key Idea:** Instead of remembering everything in one context vector, the decoder learns to also look for which words in the input to focus on while generating the output.

# Attention is All You Need

- Attention worked so well that researchers asked: **what if we remove the RNN entirely and build the whole architecture around attention?**
- In 2017, Google researchers proposed a network architecture called the ***Transformer***, which was based on an “attention” mechanism and removed the recurrence network that we saw in RNNs
- It enabled massive parallel training
- Scaled to much larger datasets
- In the Transformer, encoding and decoder components are stacked on top of each other
- Now, both the encoder and decoder blocks would revolve around attention instead of an RNN with attention features



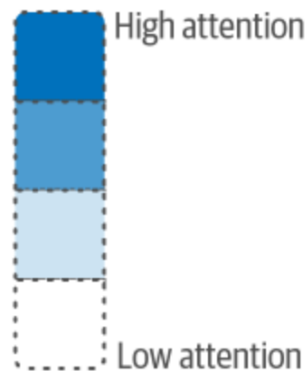
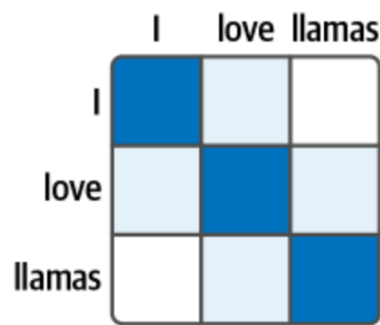
# Transformer encoder: Understanding the Input Text

- The **encoder block in the Transformer** consists of two parts, *self-attention* and a *feedforward neural network*

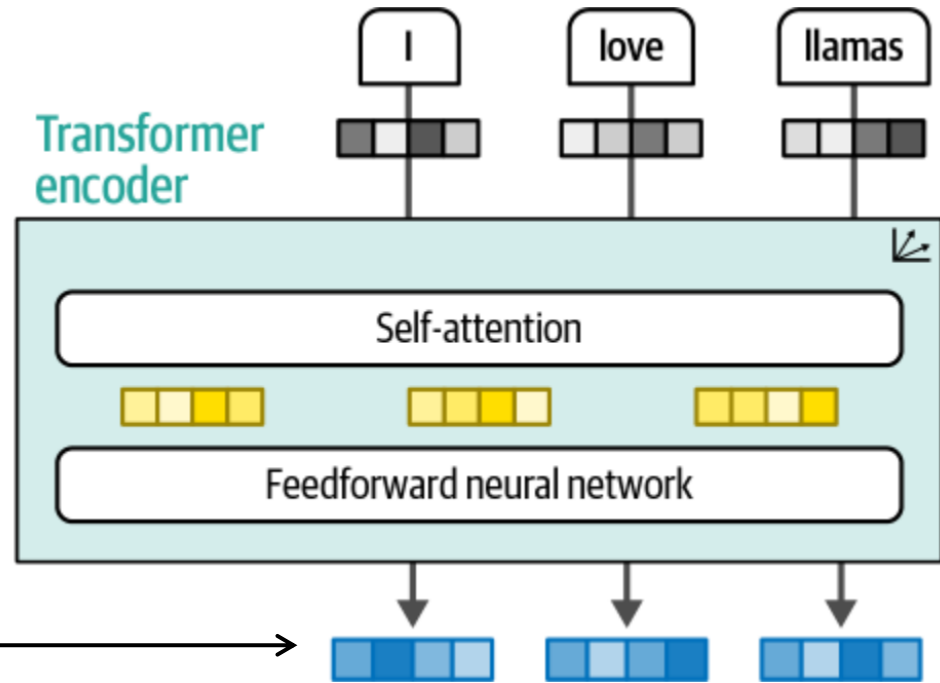
**Input:** I love llamas

- What the encoder does:**

- Reads the entire sentence simultaneously.
- Uses self-attention to determine relationships among words (tokens).
- Produces context-aware token representations.



*Self-attention attends to all parts of the input sequence so that it can “look” both forward and back in a single sequence*



"love" attends strongly to "I" and "llamas"

Every word can directly interact with every other word



# What does Self-Attention do?

- Suppose we have:

**I love llamas**

- Initially, the word2vec embedding for “llamas” represents:

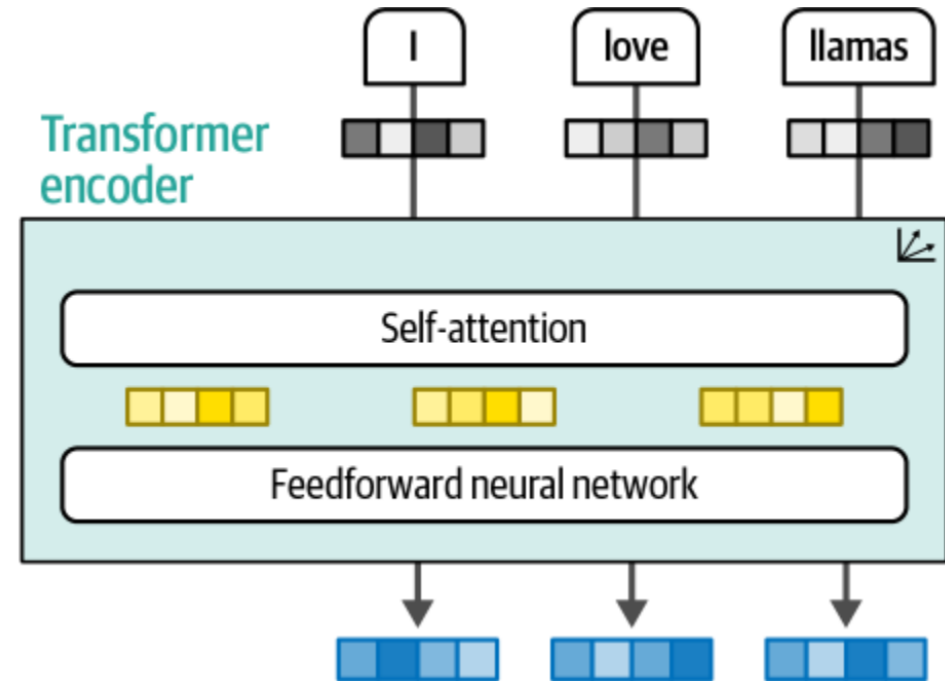
**llamas**

- After self-attention, the representation of “llamas” becomes aware of:

**I love llamas**

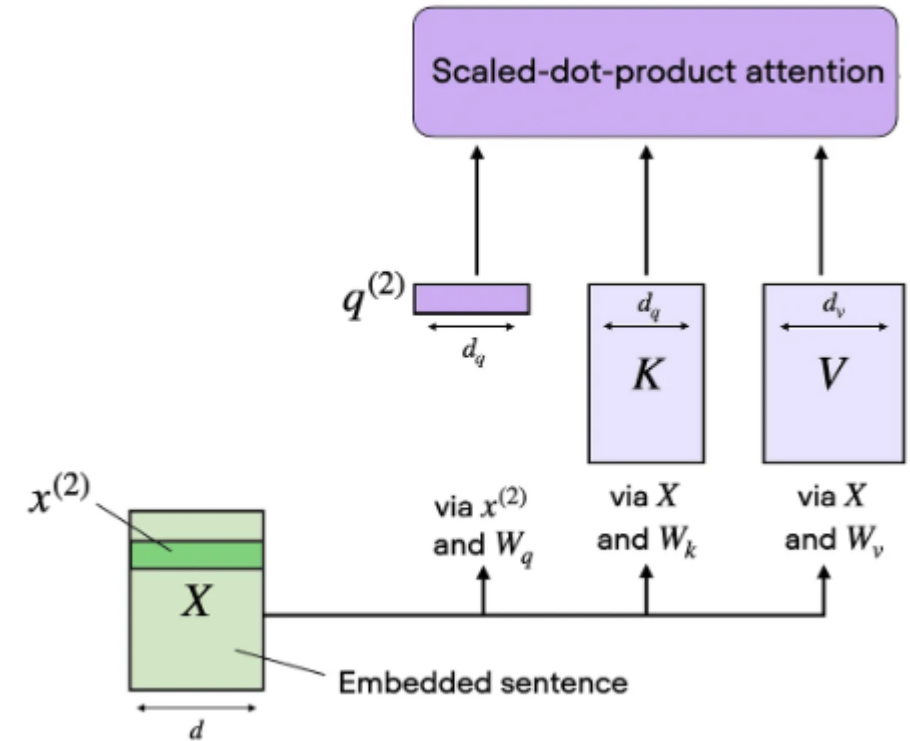
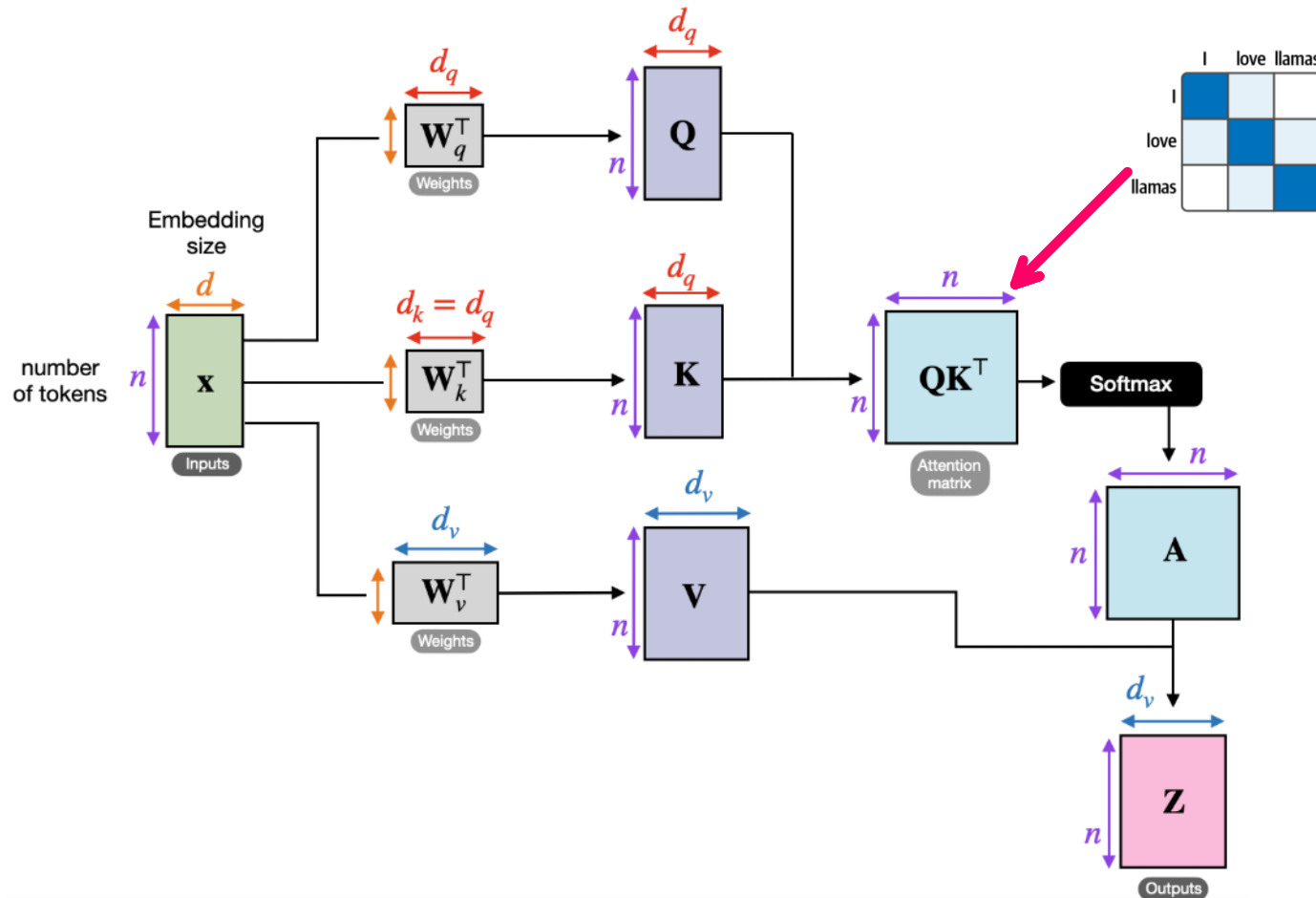
because attention has mixed information from all tokens.

- You can think of self-attention as the idea of **gathering context**



# Attention Mechanism: Query, Key, and Value (Q, K, V)

- **Query (Q):** What info a token is looking for?
- **Key (K):** What info does each token contain?
- **Value (V):** What info can each token contribute?
- **Attention Mechanism:** The model compares queries with keys to compute attention scores.
- **Output Representation:** The final output is a weighted combination of the corresponding values.



One attention head is a complete mechanism. One set of learned projections produces one attention matrix

# Understanding Attention with Search



**Query ( $Q$ )**

**Key ( $K_1$ )**

Ameca Humanoid Robot AI Platform  
3.7M views · 3 years ago  
Engineered Arts  
First look at Ameca, most advanced humanoid robot from

**Key ( $K_2$ )**

But what is a neural network? | Deep learning chapter 1  
18M views · 7 years ago  
3Blue1Brown  
Additional funding for this project was provided by Amplify Partners Typo correction  
CC  
12 chapters Introduction example | Series preview | What are neurons

**Key ( $K_3$ )**

India vs New Zealand Final ICC Champions Trophy Highlights  
#OFFICIALS 2.0  
India vs New Zealand Champions Trophy 2025 Final Match Highlights 2025  
New

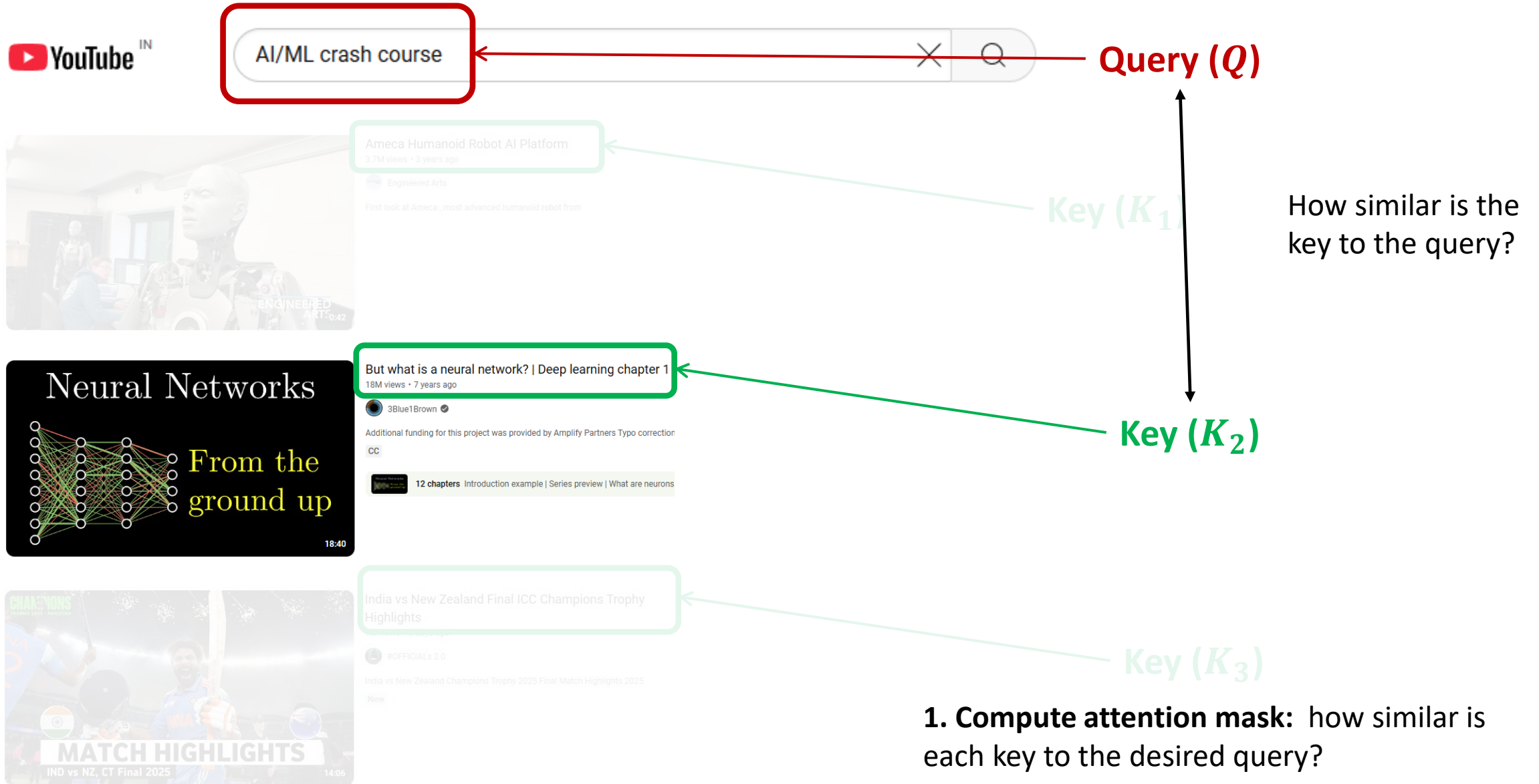
Neural Networks  
From the ground up  
18:40

CHAMPIONS  
TROPHY 2025 · PAKISTAN  
MATCH HIGHLIGHTS  
IND vs NZ, CT Final 2025  
14:06

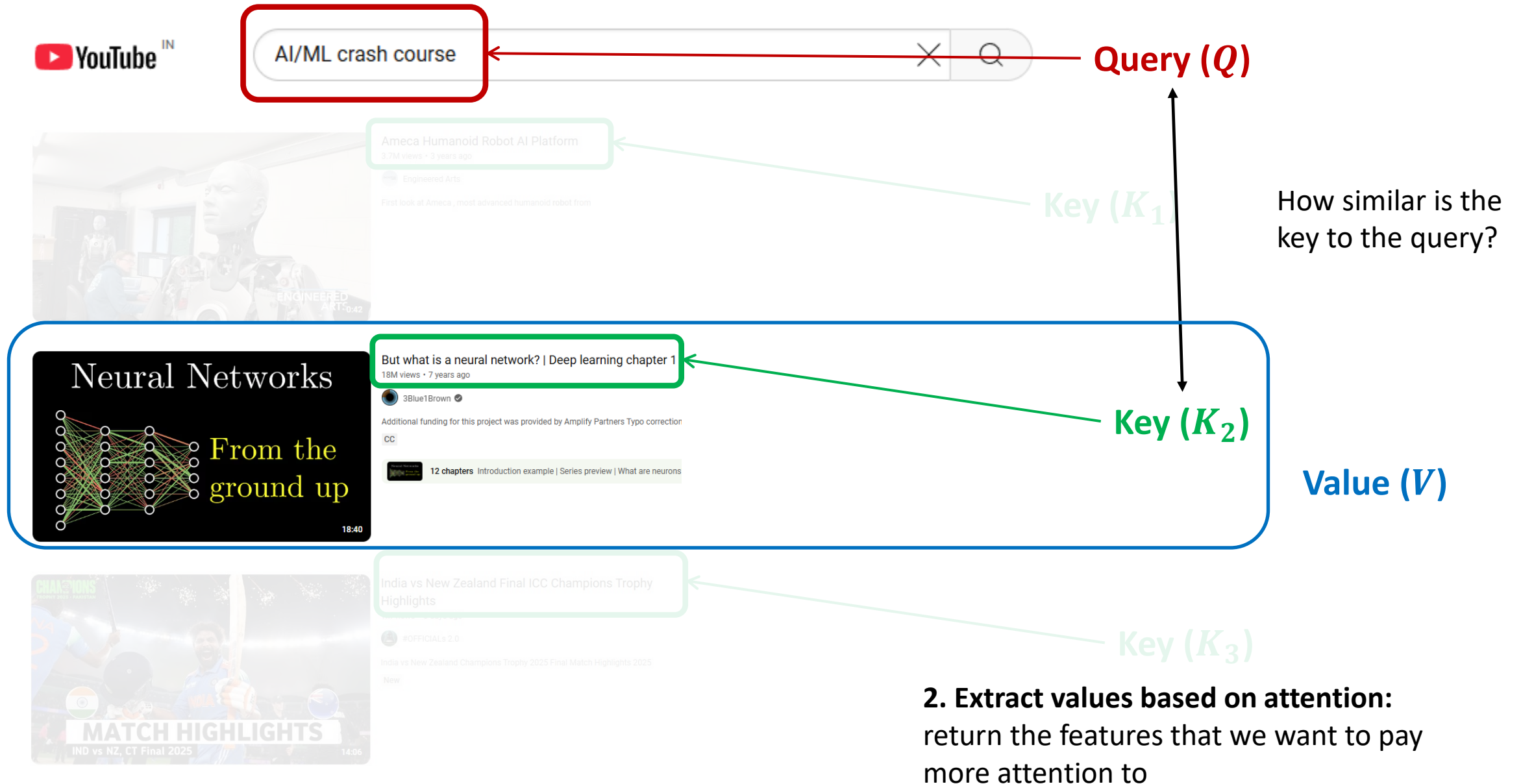
Keys are descriptors that capture/summarize the information contained in the videos

Search operates by comparing the closeness of the query with keys

# Understanding Attention with Search

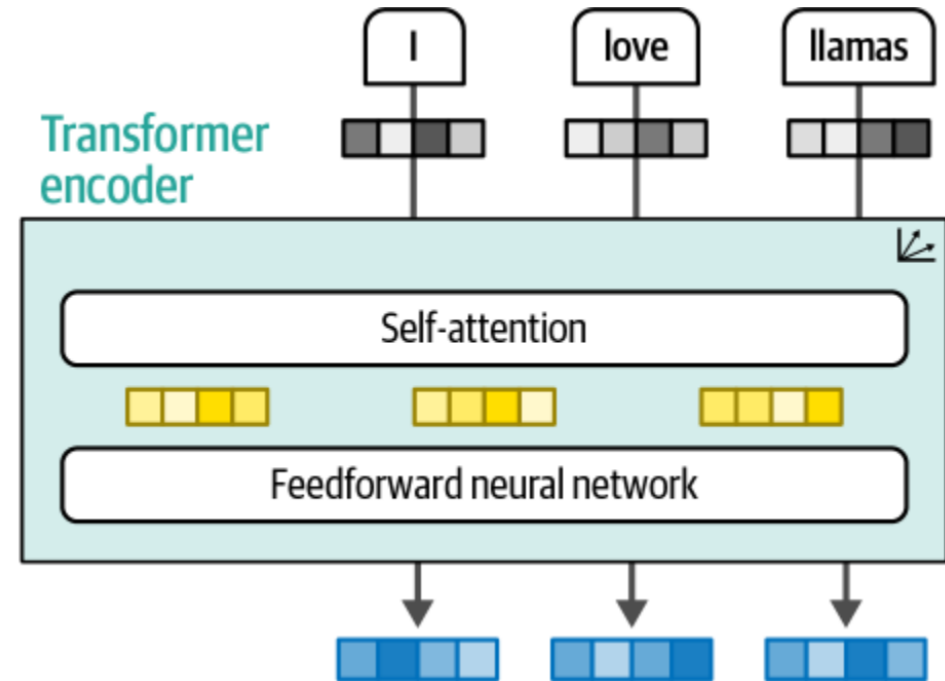


# Understanding Attention with Search



# What does the **FNN** do?

- The FFN is applied independently to each token:
  - Token 1 → FNN
  - Token 2 → FNN
  - Token 3 → FNN
- No communication occurs between tokens here.
- The FFN does **feature processing**:
  - refines the representation,
  - introduces nonlinearity,
  - extracts more useful features.
- The context comes almost entirely from **Self-Attention** because this is where tokens interact



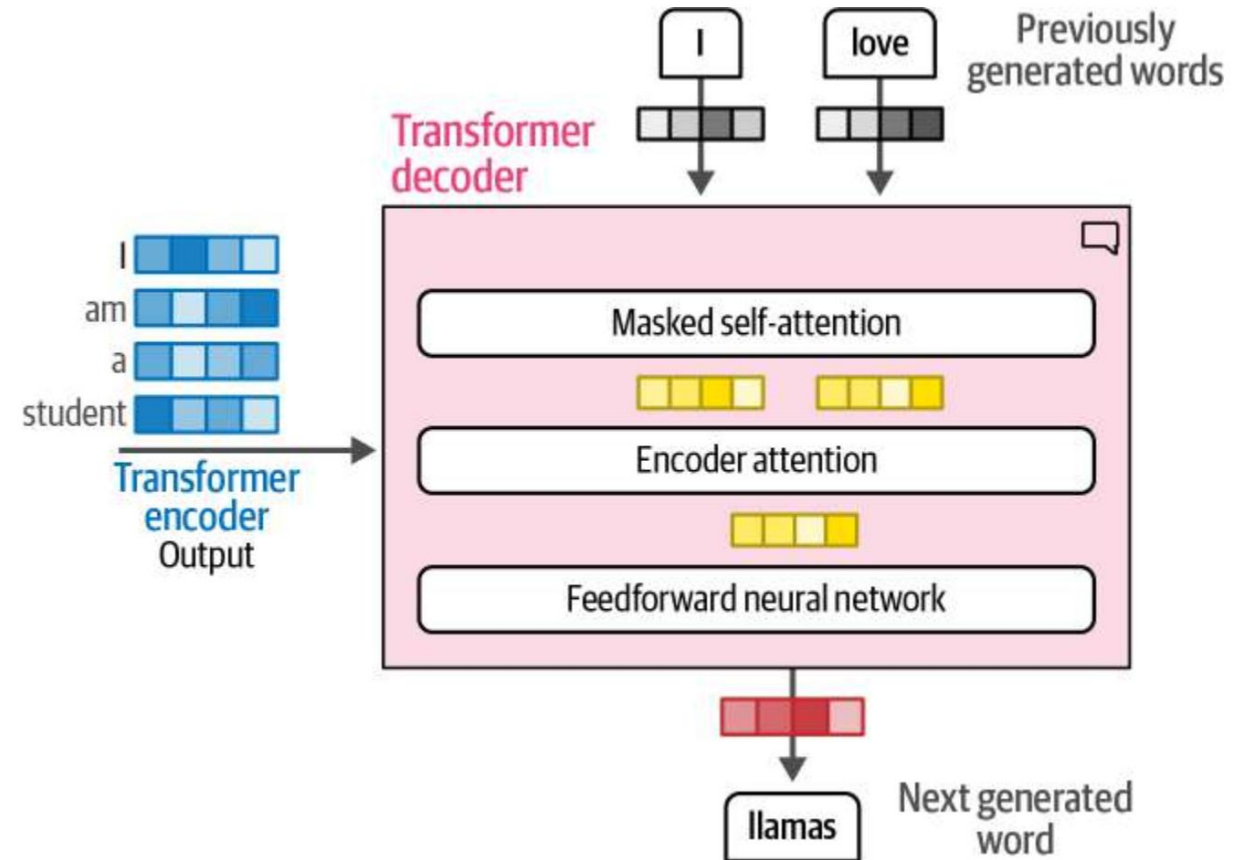
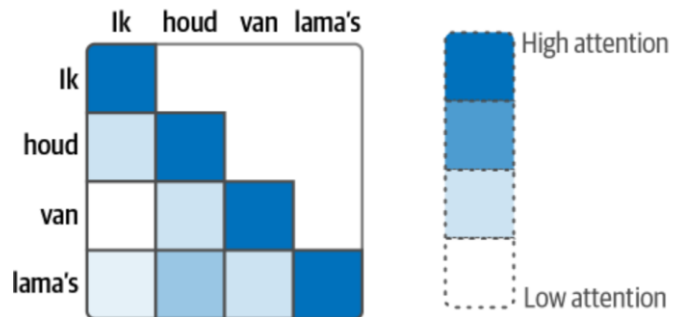
# Transformer decoder: Generating Text

- Inputs to the Decoder

1. Previously generated words
2. Encoder inputs

- Two Attention mechanisms

- Masked Self-Attention
  - Looks only at previously generated words
  - Cannot see future words
- Encoder Attention
  - Looks at encoder outputs
  - Retrieves relevant info from input sentence



The decoder decides what to write next using both the source sentence and the words generated so far.

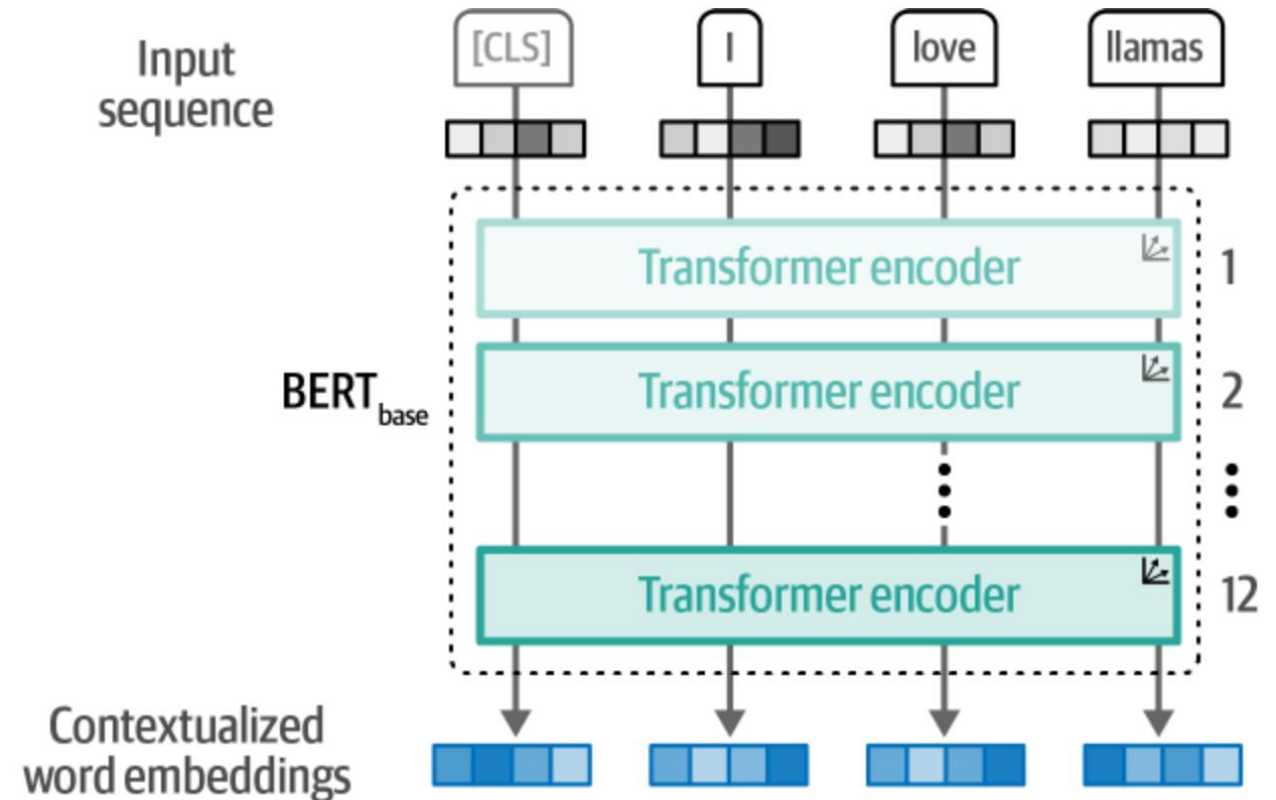


# Bidirectional Encoder Representations from Transformers (BERT)

- Original Transformer model is an encoder-decoder architecture that serves translation tasks but cannot easily be used for other tasks, like text classification
- In 2018, BERT was introduced by Google
- BERT is an **encoder-only architecture** that focuses on representing language; there is no decoder
- These encoder blocks are the same as before: *self-attention* followed by *feedforward neural networks*

Best for:

- ✓ Classification
- ✓ Search
- ✓ Information extraction
- ✗ Text generation



The architecture of a BERT base model with 12 encoders



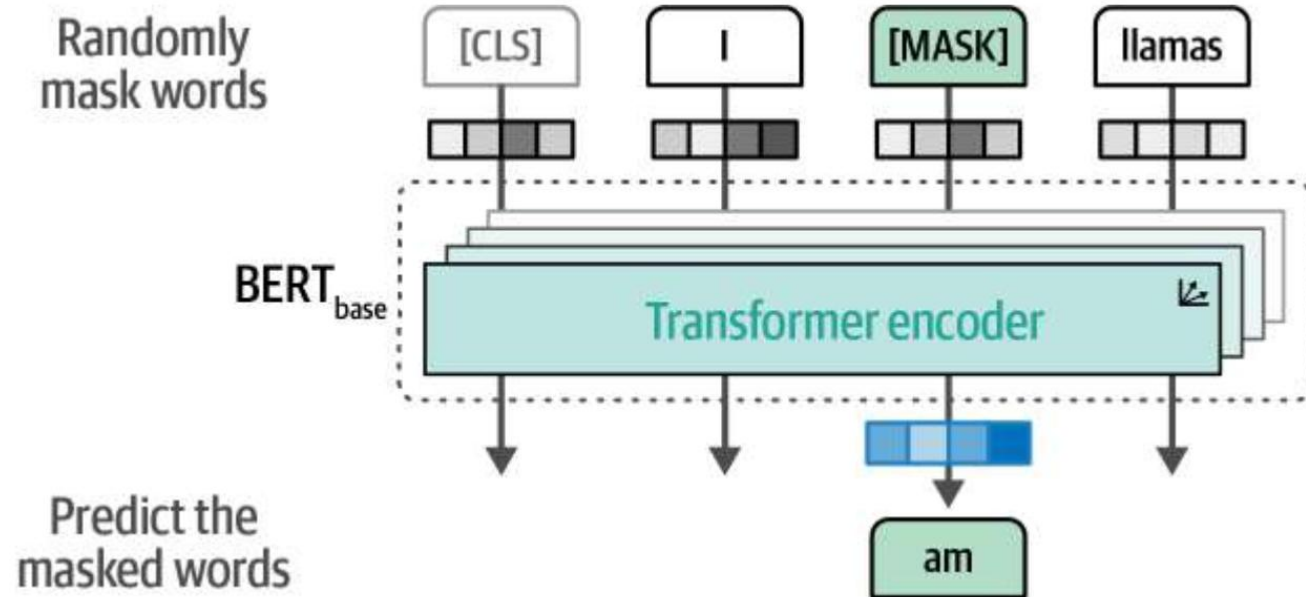
# How BERT learns: Masked Language Modeling

- Encoder stacks are trained using **masked language modeling**

- Example: I [MASK] llamas

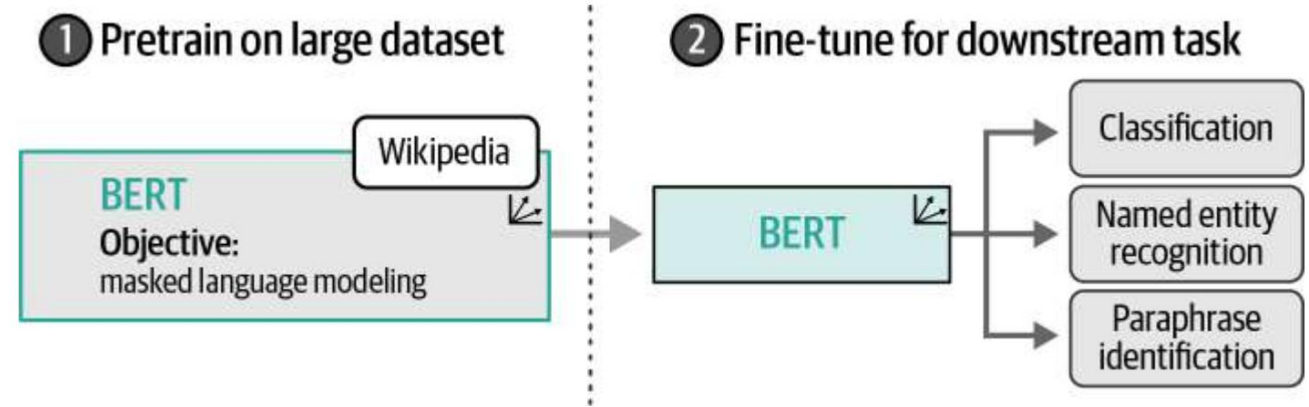
- Predict: love

- Random words are hidden.
- BERT learns to recover them.
- This forces deep understanding of context.



# Why BERT became Popular: Pretraining + Fine-Tuning

- Step 1: Train on huge amount of text (pre-training)
- Step 2: Fine-tune on specific tasks
- Examples:
  - Sentiment Analysis (e.g. Amazon reviews)
  - Spam detection
  - Document classification

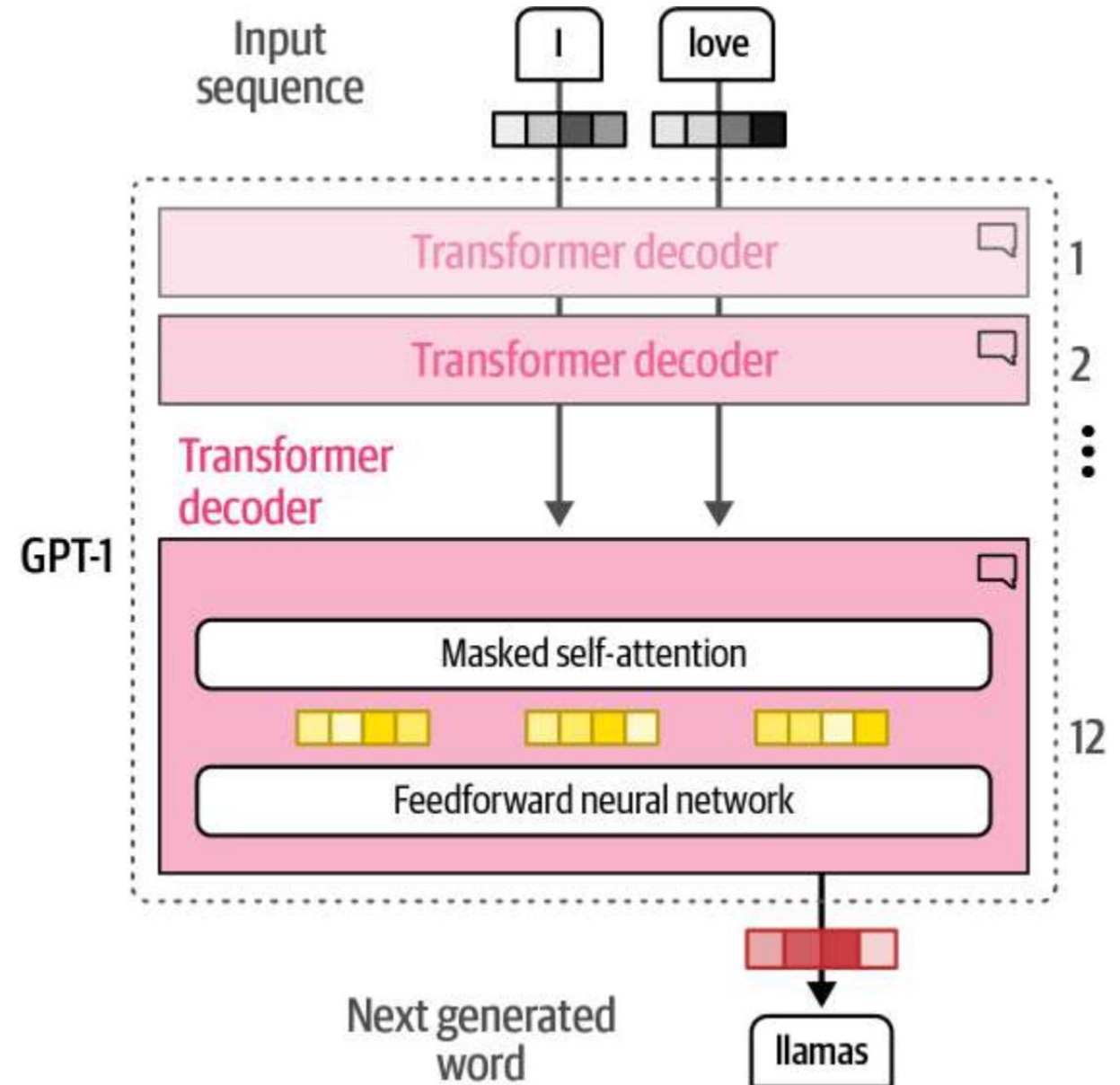
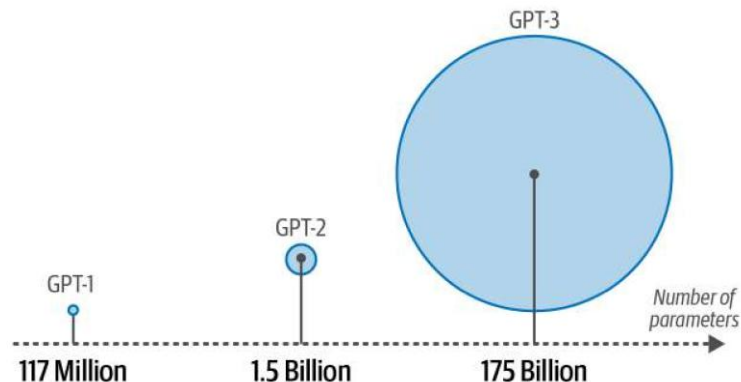


- A huge benefit of pretrained models is that most of the training is already done for us.
- Fine-tuning on specific tasks is generally less compute-intensive and requires less data.
- Encoder-only models like BERT can be referred to as **representation models**
- In contrast, decoder-only models will be referred to as **generative models**

**Key takeaway:** Reuse a pretrained language model instead of training from scratch.

# Generative models: Decoder-Only Transformers

- BERT is excellent at **understanding text**
- But BERT is not designed to **generate text**
- We need a model that can generate one token at a time
- In 2018, OpenAI introduced **Generative Pre-trained Transformer (GPT)**
  - Uses Decoder-only Transformer
  - It stacks decoder blocks
  - Learns by repeatedly predicting the **next token**
  - Designed specifically for **text generation**

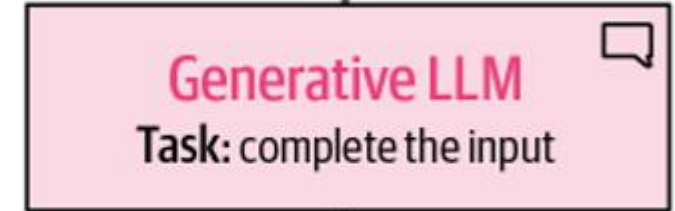


# Generative Transformers Models

- Generative Transformer models, particularly large decoder-only models such as GPT, Claude, Gemini, and Llama, are commonly referred to as **Large Language Models (LLMs)**
- These models generate text by repeatedly predicting the next token in a sequence.
- These models can take in a user query (**prompt**) and output a response that would most likely follow that prompt.

User query  
(prompt)

Tell me something about llamas



Output

Llamas are domesticated South American camelids, widely used as pack animals by Andean cultures since pre-Hispanic times. With their fluffy coat, long neck, and distinctive facial features...

GPT is trained as a next-token predictor. **ChatGPT** is GPT further trained to follow instructions and engage in conversation.

# Generative Transformers Models

- A vital part of LLMs is something called the **context length** or **context window**
- The context length represents the maximum number of tokens the model can process
- Note that due to the autoregressive nature of these models, the current context length will increase as new tokens are generated.

